Contents

The cost of post quantum cryptography on drone and robot microcontrollers: evidence, models and a migration path **1**
Abstract . . . 1
1. Introduction . . . 1
2. Related work . . . 2
3. Method . . . 3
4. Results . . . 4
5. Discussion . . . 6
6. Threats to validity . . . 6
7. Limitations . . . 6
8. Reproducibility . . . 7
References . . . 7

The cost of post quantum cryptography on drone and robot microcontrollers: evidence, models and a migration path

Author: Huda Mueen. Draft of 2026-10-20. Generated from `docs/PAPER_body.md` by `scripts/build_paper.py`; every number below is read from a file in this repository, and `scripts/check_numbers.py` fails the build when a stated number drifts from its file. Licence: all rights reserved (`LICENSE`).

Abstract

Drones and ground robots authenticate commands, telemetry and firmware with classical public key cryptography on microcontroller flight computers and narrow radio links. NIST has standardised the replacements (ML-KEM, ML-DSA, SLH-DSA), and buyers have begun to set dates. This study asks what the replacements cost on such devices and links, how well that cost can be predicted before a board is measured, and which devices in a fleet can migrate in place. It combines 1,328 published rows (measurements, sizes and protocol figures) read from 51 sources, the newest 357 each read by two independent readers, 403 earlier ones rechecked in full by a fresh reader and the rest checked on samples or by the collector, a trace driven cycle model checked on held out published rows of the same code base, cycle accurate simulation of an open source microcontroller class core, an energy and CPU headroom model built from datasheet values, a link and protocol simulator checked against a few published timings, a signed boot and update model, a fleet planner run on real flight controller classes, and a mission level simulation of four security architectures. Within one code base, a cycle model fitted on half of the published rows predicts the other half with a median error of 1.0 percent, though signing misses by up to 23 percent. On telemetry class radios a hybrid key establishment followed by a symmetric session keeps command availability where it is without authentication, while a post quantum signature on every command does not. For firmware, hashing the image costs more than any signature check unless the device has hash hardware. The largest open uncertainties are the free memory a flight stack leaves and the real timing of mesh links. Nothing in this study was measured on a microcontroller; the board experiment is prepared and its outcome predicted in advance, but it can test the model only once the kit can run with caches off.

1. Introduction

A small multirotor or ground robot runs its flight or motion stack on a microcontroller class processor and talks to its operator over a telemetry radio, a mesh network or a cellular link. Public key cryptography protects three things on such a device: the command channel (who may steer it), the update channel (which firmware it will run) and, sometimes, the confidentiality of its data. Each of these is threatened differently by a future quantum computer. Confidentiality is exposed

today if recorded traffic stays valuable for longer than the time to migrate. Authentication of commands is exposed only from the day forgery becomes possible, but a device that cannot be updated by then cannot be fixed. A boot chain whose root of trust is fixed in read only memory cannot be fixed at all.

The replacement algorithms are standardised: ML-KEM for key establishment [nist_nistpubs_fips_nist_fips_203], ML-DSA and SLH-DSA for signatures [nist_nistpubs_fips_nist_fips_204] [nist_nistpubs_fips_nist_fips_205], and the stateful hash based schemes LMS and XMSS for firmware signing [rfc-editor_rfc_rfc8391]. What an operator cannot yet say from evidence is what they cost on its own flight controller, over its own link, across its own fleet. Published microcontroller measurements are many but uneven: most come from one core (the Cortex-M4), from libraries tuned for a benchmark framework [iacr_2019_844], under clock, cache and counter settings that differ from paper to paper [iacr_2026_1333].

This study answers six questions:

- RQ1 What do the standard algorithms cost in cycles, stack and flash on microcontroller cores, and how well can cycles be predicted without a board?
- RQ2 What do they cost in energy and CPU headroom, and what share of a vehicle's budget does a realistic security workload take?
- RQ3 How do handshake, rekey, signed command and update traffic behave over the radios such vehicles use?
- RQ4 What do signed boot and signed update cost, and where does a fixed root of trust force replacement?
- RQ5 For fleets built from real board classes, which devices can migrate in place, with what probability?
- RQ6 Which architecture for an authenticated command and update channel is feasible, and what follow on project does the evidence justify?

The contribution is not a new algorithm or implementation. It is an evidence base with its provenance, a set of models whose errors are measured rather than assumed, and a pre registered, ready to run experiment that will test those models on hardware.

2. Related work

Benchmark frameworks and optimised implementations. The pqm4 framework set the common method for Cortex-M4 measurements [iacr_2019_844] and publishes a results table [github usercontent_mupq_pqm4_master_benchmarks_csv]. Later work moved optimised code to the Cortex-M7 [iacr_2025_366], reduced memory for ML-DSA [iacr_2022_323] [iacr_index_php_tches_article_download_8725_8325], and revisited Keccak and Dilithium on ARMv7-M [rub_index_php_tches_article_download_11419_10922_11688]. Measurements on small cores include the Cortex-M0+ of the RP2040 [arxiv_pdf_2603_19340]. A recent methodology paper shows that results differ widely with library configuration, caches and statistics, and proposes reporting rules [iacr_2026_1333]; its Cortex-M33 rows are used here as an independent check.

Hash based signatures and secure boot. LMS and XMSS have been compared on the Cortex-M4 [iacr_2020_470], XMSS verification has been accelerated in hardware [nsf_servlets_purl_10112210], and post quantum secure boot has been built and measured [date-conference_proceedings_archive_2020_pdf_0438]. Open bootloaders document post quantum options [githubusercontent_wolfssl_wolfboot_master_docs_pq_md] and immutable first stages [readthedocs_en_latest_design_docs_booting_bl1]; a widely used autopilot documents its own secure boot [px4_main_en_advanced_config_bootloader_secure_boot]. Software update over constrained networks with post quantum signatures has been measured on low power devices [iacr_2021_781].

Protocols and robotics. Hybrid key agreement for TLS 1.3 is specified [ietf_doc_rfc10024]. MAVLink 2 defines a message signing scheme with a shared secret [mavlink_en_guide_message_signing]. Post quantum command authentication for unmanned systems has been studied under

real time and bandwidth limits [iacr_2026_2343], and resource constrained nodes have been integrated with ROS 2 security [ipp_docs_secure_integration_of_extremely_resource_constraine]. Hardware and co design approaches appear in [iacr_2026_312] and [pqshield_wp_content_uploads_2024_10_high_performance_ntt_har].

What this study adds is the combination: one provenance controlled table across these strands, models fitted and validated against it, and fleet and mission level consequences derived from it, each with its uncertainty and its label.

3. Method

3.1 Evidence

Every number carries one of seven labels, kept apart in every table and figure: published (copied from a retrieved source with its location), host measured (run on a PC), emulated (run on QEMU or Renode models; correctness, stack and size only), estimated (arithmetic from published sizes or rates), modelled (trace driven or fitted model, with its validation error beside it), RTL simulated (cycle accurate simulation of an open source core design, not Arm silicon) and planned (not yet run). A board measured label is reserved for the board experiment.

Rows enter `data/evidence.csv` only through staging files and a merge script that changes no value. Earlier rounds verified rows by independent readers on random and headline samples (details in `docs/VERIFICATION.md`, `docs/VERIFICATION2.md` and `docs/VERIFICATION3.md`) and rechecked 403 rows from supplied papers in full. In this round every new row was read by two readers who saw neither the staged value nor each other's sheet; every disagreement on a value, and a seeded sample of disagreements on wording, was settled by a third look (8 of 439 staged rows with values had a wrong value before correction, 1.8 percent, Wilson interval 0.9 to 3.6 percent; `docs/VERIFICATION5.md`). Searches are logged in `docs/SEARCH_PROTOCOL.md`.

3.2 Models and simulations

Where a model is fitted, the split is seeded and recorded and the error on the held out part is reported before any modelled value is used. Not every model could be validated: the cycle model is checked on held out rows of the same code base; the energy model is calibrated on one board and one source; the boot model rests on three held out rows; the link model's rate part is checked against one published table, its LoRa formula against published examples and its duty cycle and loss parts only against closed forms; the protocol, mission and fleet results have no measured data to be checked against and are estimates under stated assumptions. The cycle model counts executed instructions by class in a trace of the pinned benchmark code and maps them to cycles with either per class costs assumed by the author (the Arm timing tables could not be retrieved) or costs fitted on half of the matched published rows. The energy model multiplies cycles by datasheet run mode current and voltage. The link model and protocol simulator use cited radio parameters, closed form checks and Monte Carlo runs with intervals. The planner classifies devices against published memory footprints; the exposure model draws data lifetime, migration time and policy horizon from stated ranges. Variance based sensitivity indices name the input that matters most.

3.3 Pre registration and review

Predictions with refutation clauses were hashed before each round and are scored by script; misses stay (`docs/PREDICTIONS.md`, `docs/PREDICTIONS_V2.md`, `docs/PREDICTIONS_V3.md`, `docs/PREDICTIONS_V4.md`). Independent reviewers who saw only the repository reviewed each round; their findings and resolutions are logged.

4. Results

4.1 RQ1: compute cost, and how well it can be predicted

Evidence. The table holds 1,328 rows; 722 are microcontroller rows with a cycle count, on 8 core labels as the sources print them (Cortex-M4 and Cortex-M4F are counted apart, and one label is ‘not stated’). Inside groups of the same parameter set, operation and core class that have rows from two or more sources, the largest published value is 1.46 times the smallest at the median group (up to 4.67; `docs/META_ANALYSIS.md`). Part of this spread is deliberate: a group can hold speed and stack optimised variants of one scheme and both means and medians, so it measures how far a published figure can sit from the deployed code when the variant and conditions are not known, not the measurement error of one implementation. Library, optimisation, toolchain, cache and statistic all move it.

Prediction without a board. A trace driven model counts executed instructions by class in the pinned pqm4 code (emulated) and maps them to cycles (modelled; `docs/CYCLE_MODEL.md`). Fitted on half of the matched published rows and tested on the other half, split by binary so that no test row shares a binary with a training row, its median absolute error is 1.0 percent (95 percent interval 0.7 to 1.7, too narrow because rows cluster by binary) on 27 held out rows, and 0.8 percent without signing; signing misses by up to 22.8 percent because its cost varies from message to message. The two halves share algorithms and much code (for example the same parameter set in two assembly variants), so this is a test of interpolation within one code base, not of transfer to new code. Without fitting, a model built from assumed per instruction costs is 12.6 percent high at the median. Applied without refitting to an independent low memory library, key generation lands 2.9 to 8.3 percent above its published Cortex-M33 values, a weak check because core, memory system and compiler all differ. The model’s predictions for the first board session are registered with intervals in `docs/PREDICTIONS_V4_BOARD.md`.

Cycle accurate simulation of an open source core. The portable C code of mlkem-native and mldsa-native was run on a cycle accurate simulation of the open source Ibex RV32IMC core (small configuration, memory without wait states; RTL simulated, `docs/RTL_SIM_RESULTS.md`): ML-KEM-768 decaps 3,033,704 cycles; ML-KEM-512 decaps 1,915,739 cycles; ML-DSA-44 verify 5,405,374 cycles; ML-DSA-44 sign 12,553,829 cycles (signing is the median of five messages and varies several fold). These are measurements of a core design in simulation, not of Arm silicon. They are 1.24 to 2.99 times the published portable C rows for the Cortex-M4 (different library, compiler, instruction set and statistic, so only the order of magnitude is compared) and give a microcontroller class reference that no vendor setting can change.

4.2 RQ2: energy and CPU headroom

Energy per operation was estimated from published cycles and cited run currents of five parts and checked against published energy rows: uncalibrated, the estimate is low by a median of 54.8 percent on the held out rows; after one scale factor fitted on the other half, the median error is 0.7 percent (`docs/ENERGY_STUDY.md`). The check uses one board and one source, so it calibrates a factor rather than validating the model in general, and the energy figures elsewhere in this paper are uncalibrated; on that board the published energy is 2.21 times the estimate, so they are likely low by a similar factor; the message rates below depend only on cycles and are unaffected. At 20 percent CPU headroom a 168 MHz Cortex-M4 flight controller can verify about 23.6 ML-DSA-44 signatures per second, 46.5 Ed25519 signatures and 12.6 LMS signatures, against about 1,377 messages under a symmetric MAC session (estimated). Energy per hour is small for every design; CPU time, not battery, is the binding budget, and the command rate is the input that matters most (Sobol indices in `energy/results/sobol.csv`).

4.3 RQ3: links and protocols

The link model was repaired one cited cause at a time and checked against published figures and closed forms (`docs/LINKSIM_STUDY.md` section 4.4). The LoRa time on air formula reproduces

the published examples, but this is a weak check: only one of the four examples states every parameter, and for the other three the unstated settings were chosen after the fact because they reproduce the published value. Fragment counts agree with an independent count and the duty cycle wait equals its formula, an arithmetic check; stop and wait retransmissions in the event simulation agree with their closed form within Monte Carlo intervals, but selective repeat, which the protocol and mission studies use, is not checked. For IEEE 802.15.4, adding every cited cost (PHY header, 6LoWPAN fragment headers and alignment, acknowledgements, inter frame spacing, clear channel assessment and backoff, turnaround) takes a 50 kB transfer from 4.5 s to 7.1 s, which is still 4.2 times faster than the published 30 to 60 s. Most of that gap is unexplained by single hop costs; four hops would bring the model to 28.5 s, near the low end, but the source gives no hop count, so the model is reported as optimistic on 802.15.4 networks. The rate part was checked against one published table of handshakes over an emulated link, with one constant fitted on two of its rows; that is a weak test on one testbed.

The protocol simulation (1000 seeded trials per cell, `docs/PROTOCOL_SIM.md`) gives the time to the first authenticated message. On a telemetry radio and on 802.15.4, a hybrid session handshake completes in about one to two seconds; on LoRaWAN it takes minutes at DR5 and hours at DR0, because of the duty cycle. Per message post quantum signatures overload the slower links.

4.4 RQ4: signed boot and update

For a 1 MB image on a 168 MHz Cortex-M4 (the STM32F405 class of a small drone flight controller) with software SHA-256, verification takes about 462 ms with ML-DSA-44 and 470 ms with LMS, of which hashing the image is 454 ms. With a hash peripheral, which the STM32F405 itself lacks (the rows are a what if for a part of that class with one), the totals fall to 23 ms and 30 ms; the signature then becomes the larger part for the slower verifiers, while for Ed25519, ML-DSA-44 and ML-DSA-65 hashing still costs about as much as the signature or more. The choice of signature scheme changes boot time far less than image size and hash hardware do (`docs/BOOT_UPDATE_STUDY.md`). Against published bootloader timings the uncalibrated model is far off and the calibrated check rests on three rows, so these are estimates of order, not predictions. A fixed first stage root of trust that verifies only a classical signature cannot be changed by any of this; such devices need hardware replacement for signing migration.

4.5 RQ5: fleet migration

The round 3 synthetic fleet is reproduced exactly (0.260 to 0.382 of devices need hardware replacement). On fleets drawn from nine real board classes, with every memory threshold drawn from published rows, the median share is 0.201 (2.5 to 97.5 percentile 0.186 to 0.232). The share is close to 0.20 and its intervals cross that value, so whether fewer than a fifth of devices need replacement depends on the threshold scenario. No retrieved page states how much flash or RAM a flight stack leaves free, so the free share is an assumption, and it moves the result more than the choice of implementation does (`docs/PLANNER_STUDY.md` section 5). Under the Mosca inequality with stated ranges, data lifetime has the largest total Sobol index (0.91), largely because its range is the widest. Policy dates come from primary texts (`docs/POLICY_TIMELINE.md`); the supplied UAE policy texts give no post quantum date.

4.6 RQ6: mission level design

In the mission simulation (1000 runs per configuration, `docs/MISSION_STUDY.md`), a classical session, a hybrid session and hash signed firmware with session keys keep the drone's control link available about as often as an unauthenticated stream on a telemetry radio (mean over runs 0.992 for the hybrid session): link outages, not authentication, set availability. A post quantum signature on every command adds about 380 ms per command on that radio, lowers availability to 0.794, and overloads the link for a ground robot streaming commands at 5 Hz (utilisation 1.88). LoRaWAN carries none of the command streams within its duty cycle. The resulting reference architecture (`docs/REFERENCE_ARCHITECTURE.md`): hash based signatures (LMS) for firmware, a

hybrid key establishment followed by a symmetric MAC session for commands on telemetry class links or faster, and no per command signatures on streamed commands.

5. Discussion

Three results change what an engineer should do. First, a published cycle count is not a property of the algorithm alone: the implementation variant and the measurement conditions move the same operation on the same core class by a factor that the meta analysis quantifies, so a migration decision should rest on a measurement of the deployed code, which the board kit makes cheap. Second, on the links drones and robots use, the cost of post quantum authentication is a one time handshake, not a per message burden, provided the design uses a session: per message signatures are the only option that fails. Third, the fleet answer depends more on how much memory the flight stack leaves free than on which implementation of ML-DSA is chosen; operators can settle this from their own build output, which no public page states.

Where the models overlap they are not independent: the mission model reads the link simulator's parameters, so their agreement on a single handshake is a consistency check, not a validation, and the two still differ noticeably on the time to set up a hybrid session on the telemetry radio, a difference this round did not explain. The energy and mission studies both find CPU time, not energy, to be the binding budget. They disagree with the evidence in one place that matters, the 802.15.4 transfer time, and the study reports that gap rather than closing it by fitting.

6. Threats to validity

Internal validity. The cycle model is fitted to one framework's code and checked on its other half; every test binary has a sibling of the same implementation in training, so agreement there shows interpolation within one code base, not transfer to other libraries; its bootstrap intervals treat rows as independent although they cluster by binary, so they are too narrow; and its external check on a second core gives two distinct predictions. The fitted per class costs are not physical cycle costs. The link model's absolute times are checked against few published figures. Energy uses typical datasheet currents, not measured ones.

External validity. Published cycle counts come from development boards at benchmark clocks with caches and wait states set by each author; a flight controller running its flight stack has less free memory, interrupts and bus contention. Fleets in the planner are drawn from real board classes but their counts are synthetic.

Construct validity. Stack figures follow different definitions across sources; signing times are variable and summarised by different statistics.

Reliability. Every result regenerates from committed scripts with stored seeds; continuous integration fails when a document disagrees with its data.

7. Limitations

1. Nothing was measured on a microcontroller. The board experiment is prepared and its outcome registered, but not run, and the kit cannot yet switch caches off, which the registered predictions assume (`docs/PREDICTIONS_V4_BOARD_NOTES.md`).
2. The cycle model is validated within one code base; its transfer to other libraries rests on a single small external check.
3. The link model is too fast on IEEE 802.15.4 by a factor the study cannot yet explain; duty cycle and fragmentation are checked only arithmetically, and loss handling only against closed forms for stop and wait, with no measurement.
4. Fleets are drawn from real board classes but their counts and free memory shares are synthetic or assumed.
5. Energy rests on typical datasheet currents for five parts and one calibration source.

6. The classical baselines (Ed25519, X25519) are single published rows chosen within a published range that spans about a factor of two; the energy and mission studies use the same rows and show the range beside each result.
7. The studies do not yet share one set of inputs everywhere: the protocol and mission models describe the MAVLink style handshake differently, and the energy and mission models use different published LMS rows and per message costs; each choice is stated in its study.
8. Several sources could not be retrieved (listed in `docs/NEEDS_FROM_AUTHOR.md` and the source notes); 126 new rows that only one reader could confirm were left out of the table.
9. The double reading settled every value disagreement but only a sample of wording disagreements, so some operation, scheme or library fields are likely still wrong (`docs/VERIFICATION5.md`, Limits); two readers shared a scratch path for helper scripts, a limit on their independence.

8. Reproducibility

Run `python3 scripts/merge_evidence.py`, then the `run_all.py` script of each model folder (`cyclemodel/`, `energy/`, `bootchain/`, `mission/`), `rtlsim/fetch.sh`, `rtlsim/build_sim.sh`, `rtlsim/build_sw.sh` and `rtlsim/run.sh`, the link study scripts in `linksim/`, the planner study in `planner/`, and `python3 scripts/build_paper.py`. Third party libraries are fetched at pinned commits and never committed. Data dictionary: `docs/DATA_DICTIONARY.md`.

References

Keys refer to `data/bibliography.bib`, generated from the evidence table and the source notes.